TIRTH NOW · BRAJ SPIRITUAL TRAVEL

Phase 2 — Backend Modules Completion Report

Bookings, payments, safety, realtime, reviews, reels, AI guide and admin — 16 modules, 227 routes

STATUS	COMPLETED	PREPARED BY
Complete	11 October 2026	Vivek Shukla
BRANCH	**COMMITS**	**NEXT**
claude/peaceful-fermi-hvmzt4	7436fc7 … 88673c4 (9 commits on top of Phase 1)	Phase 3 — Mobile app (Flutter)

1. Summary

Phase 2 delivers the whole backend the apps and portals will use: 16 domain modules, 227 HTTP routes, a Socket.IO gateway, background workers and a double-entry money ledger. Bookings, payments, refunds and vendor payouts work end to end; travellers get SOS alerts, notifications, reviews, reels and an AI guide that never invents aarti times.

Like Phase 1, everything runs locally with **zero external accounts**: every third-party service (Razorpay, FCM, MSG91, WhatsApp, Claude, OpenAI embeddings, Google speech and vision, Cloudflare Stream/Mux, OpenWeather, maps, transit) sits behind an interface with a real adapter and a deterministic mock.

Verification at sign-off

Run on the final commit against PostgreSQL 16 + pgvector 0.6 and Redis 7.

Check	Command	Result
Formatting	`pnpm format:check`	✓ clean
Lint (type-aware, strict, module boundaries)	`pnpm lint`	✓ 0 errors, 0 warnings
Type-check	`pnpm typecheck`	✓
Unit tests	`pnpm test`	✓ 326 passed (API 322, shared-types 4)

Check	Command	Result
API end-to-end tests	`pnpm test:e2e`	✅ 123 passed in 15 suites
Migration drift	`pnpm --filter @tirth-now/api db:check`	✅ in sync
Seed (run twice)	`pnpm db:seed`	✅ idempotent
Production build	`pnpm build`	✅ API and worker boot from `dist/` (worker serves health only, no socket server)

2. Scope delivered

Module	Highlights
core	Idempotency-Key interceptor, BullMQ queues with role-aware workers, recurring maintenance tasks, Redis cache, settings, audit log, field encryption, circuit breaker
identity	Profile, devices, emergency contacts, account deletion; admin user search, suspension, platform roles
files	Presigned uploads, content sniffing, WebP variants, generated files
catalog	Cities, temples, aarti RRULE expansion with seasons, festivals, POIs, safety directory, explore, parikrama routes, search, admin CMS
vendors	Onboarding, KYC review, staff permissions, encrypted bank accounts
inventory	Hotels with 365-night room inventory, cabs (metered/fixed fares), restaurants, experiences with slots
orders	One state machine for all booking types, holds, cancellation policies, vendor actions, cab tracking
payments	Razorpay orders/webhooks/refunds/Route, commission rules, GST on commission, ledger, payouts, PDF statements
notifications	Push/SMS/WhatsApp/email, en+hi templates with DB overrides, preferences, in-app centre, aarti reminders, broadcasts
safety	SOS with contacts fan-out, live-location share page, admin board
realtime	Socket.IO `/rt` with Redis adapter: orders, vendors, admin, aarti countdowns, SOS share rooms
weather / transit	5-day forecast with Braj travel advisories; train discovery with IRCTC deep links
reviews	Verified reviews, ratings, vendor replies
reels	Transcoding, ranked feed, likes/saves/views/comments, follows, reports and moderation queue
ai	RAG knowledge base, streaming chat with DB tools, trip planner, voice, landmark recognition, quotas
admin	KPIs, runtime config, broadcasts, audit log

Endpoint reference: `docs/API.md`. Integrations: `docs/INTEGRATIONS.md`.

3. What was built

3.1 Bookings and money

- `OrdersService.transition()` is the only writer of order status: row lock, actor rules, effects (ledger sale, holds release, refunds, settlement), history row, events after commit.
- Inventory holds are taken inside the order transaction with row locks; unpaid holds expire every minute, and a payment that arrives after expiry is refunded automatically.
- Ledger accounts and postings are in ADR-0009: sales go to `vendor_pending`, settle to `vendor_payable` on completion, refunds re-split what the platform keeps (penalties are commissioned like sales). A deferred trigger keeps every transaction balanced.
- Payouts: cleared balance after a settlement hold, admin approval, Razorpay Route transfer, linked account created lazily; failed refunds can be retried by an admin.

3.2 Safety and notifications

- SOS is idempotent, rate-limited (always audited), reuses an active SOS, alerts contacts by SMS, WhatsApp and push within the job (queue fallback for failures), shares a live-location page, and tells contacts when the traveller is safe.
- Notification rows per user and channel; dead FCM tokens are removed; preferences filter by category; safety and emergency broadcasts always go out.

3.3 Realtime

- Clients authenticate in the handshake; every room subscription is authorised against the database. All emits go through the Redis emitter so the worker publishes too.

3.4 Social content

- Reels transcode after the upload passes verification; feed ranks by a decaying hot score with keyset cursors. Captions, comments and reviews are moderated; three distinct reports hide content until a moderator acts; one decision closes every report on it.

3.5 AI guide

- Claude (`claude-opus-5-5`, cheap tier `claude-haiku-5-5`) through the official SDK, with refusal fallback, strict tools and structured output (ADR-0015).
- Aarti times come only from the `get_aarti_schedule` tool or the planner's database candidates; the planner's output is validated with the shared Zod schema, retried once, then falls back to a deterministic plan.
- Only admin-approved documents are indexed (pgvector HNSW); answers cite sources.

4. Testing

Suite	Tests
health, auth, core, files, identity, catalog, vendors, inventory	83

Suite	Tests
orders (bookings, webhooks, holds, cancellations, payouts, ledger balance)	10
payments (commission specificity, slow and failed refunds)	3
safety (SOS fan-out, share page, rate limit, admin)	5
notifications, realtime, weather, transit	5
reviews, reels, moderation	5
AI (RAG, chat SSE, tools, guardrail, cache, quota, planner, voice, landmark)	7
admin (KPIs, users, config, broadcasts, templates)	5

Unit tests cover pure logic: order state machine (all transitions), cancellation policy, sale split, fares and opening hours, aarti rules, chunking, hashtags, hot score, plan validation, provider request mapping (Razorpay, MSG91, WhatsApp, OpenWeather, video webhooks, Claude adapter with a stubbed client).

Every e2e money test asserts the ledger nets to zero.

5. Running it locally

```
cp .env.example .env
pnpm install
pnpm setup                  # infra up + migrate + seed
pnpm dev                    # API on :4000 (Swagger at /docs)
pnpm --filter @tirth-now/api start:worker   # after pnpm build: queues and schedulers
```

Useful mock helpers in development: `POST /v1/dev/payments/checkout` to pay for an order, `GET /v1/dev/outbox` for sent email/push/SMS/WhatsApp.

6. Decisions recorded

ADR	Decision
0009	Double-entry ledger, settlement and refunds
0010	Commission rules and GST on commission (tax-inclusive prices)
0011	Payout eligibility and approval
0012	Train discovery with deep links, mock provider until a partner is chosen
0013	Realtime fan-out and notification delivery
0014	Vendor roles reach the client on the next token refresh
0015	AI guide, trip planner and AI providers
0016	Support tickets and home feed endpoint deferred

7. Known limitations and deferred items

- Transit has no real data partner yet (mock timetable, ADR-0012).
- MSG91 / WhatsApp delivery-report webhooks, Sentry wiring and load tests are Phase 6.
- Support tickets, `/home` aggregate and saved temples are deferred (ADR-0016).
- Real adapters (Razorpay, FCM, MSG91, Meta, Claude, OpenAI, Google, Cloudflare/Mux, OpenWeather) are unit-tested for request/response mapping but need sandbox accounts for an end-to-end smoke test (see `docs/CLIENT_SETUP_TODO.md`).
- Seeded aarti times are realistic samples and must be verified with temples before launch.

8. Actions needed from the product owner

Accounts and keys in `docs/CLIENT_SETUP_TODO.md` (★ items have long lead times: Razorpay Route, TRAI DLT templates, WhatsApp template approval), plus the content tasks listed there: approve the catalog knowledge sync, add a global commission rule, set helplines.

9. Next: Phase 3

Flutter app against this API: authentication, home (aarti, weather, festivals), temple pages with live countdowns, bookings with Razorpay checkout, SOS with the share link, notifications, reels and the AI guide (SSE chat, voice, planner).